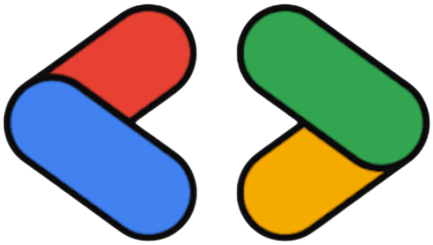
CyberSecurity Final Projects - GDGoC USU
CTF Write Up





Oleh : Naulia Situmeang

Rabu, 8 Juli 2026

Table of Content

Table of Content **1**
Summary **3**
Challenge 1 : The Secure Vault (Web Exploitation) **4**
1.1 Challenge Description 4
Gambar 1.1 - Tampilan soal "The Secure Vault" pada CTF platform 4
1.2 Methodology & Analysis 4
1.3 Tools & Commands Used 7
Gambar 1.3.1 - Hasil inspect 7
Gambar 1.3.2 - Generic payload SQL injection 8
Gambar 1.3.3 - Halaman burp suite 8
Gambar 1.3.4 - Method Not Allowed halaman login 8
Gambar 1.3.5 - Input Payload gagal 9
Gambar 1.3.6 - Tampilan gagal login 9
1.4 Flag 11
Challenge 2 : Layered Secrets (Cryptography) **11**
2.1 Challenge Description 11
Gambar 2.1 - Tampilan soal "Layered Secrets" pada CTF platform 11
2.2 Methodology & Analysis 12
2.3 Tools & Commands Used 14
Gambar 2.3.1 - Tampilan challenge.txt 15
Gambar 2.3.2 - Hasil identifikasi payload awal pada caesarcipher.org 15
Gambar 2.3.3 - Hasil decode From Hex pada CyberChef 16
Gambar 2.3.4 - Hasil identifikasi encoding Base64 pada caesarcipher.org 16
Gambar 2.3.5 - Hasil decode From Base64 pada CyberChef (bytes non-printable 17
Gambar 2.3.6 - Hasil XOR Brute Force pada CyberChef, menampilkan key 0x42 17
2.4 Solver Script 18
Gambar 2.4.1 - Script Python solve_crypto 19
2.5 Flag 19
Gambar 2.5.1 - Flag challenge 2 19
Challenge 3 : The Password Checker (Reverse Engineering) **19**
3.1 Challenge Description 19
Gambar 3.1 - Tampilan soal "The Password Checker" pada CTF platform 19
3.2 Methodology & Analysis 20
3.3 Tools & Commands Used 22
Gambar 3.3.1 - Hasil pengecekan tipe file menggunakan command file challenge 22
Gambar 3.3.2 - Hasil pencarian string pada binary, ditemukan fake flag 23

Gambar 3.3.3 - Tampilan CodeBrowser setelah proses auto-analysis pada Ghidra..... 24
Gambar 3.3.4 - Pseudo-code hasil dekompilasi fungsi main() pada Ghidra, menampilkan array local_97 dan local_b8..... 25
Gambar 3.3.5 - Solver script pada VSCode beserta hasil output di terminal..... 25
3.4 Solver Script.....26
Gambar 3.4.1 - Script python solve_reverse.....27
3.5 Flag..... 27
Gambar 3.5.1 - Flag challenge 3.....27
Challenge 4 : Hidden Pixels (Forensics).....27
4.1 Challenge Description.....27
Gambar 4.1 - Tampilan soal "Hidden Pixels" pada CTF platform..... 27
4.2 Methodology & Analysis.....28
4.3 Tools & Commands Used..... 30
Gambar 4.3.1 - Tampilan challenge.png saat dibuka menggunakan xdg-open..... 30
Gambar 4.3.2 - Hasil exiftool, ditemukan fake flag pada field Description.....31
Gambar 4.3.3 - Error "file format not supported" saat menjalankan steghide pada file PNG..... 31
Gambar 4.3.4 - Proses instalasi zsteg menggunakan gem zsteg.....32
Gambar 4.3.5 - Hasil scan zsteg dengan filter grep menampilkan flag asli pada kombinasi b1,b,lsb,xy.....33
4.4 Flag..... 33
Gambar 4.4.1 - Flag challenge 4.....33
Daftar Pustaka..... 34

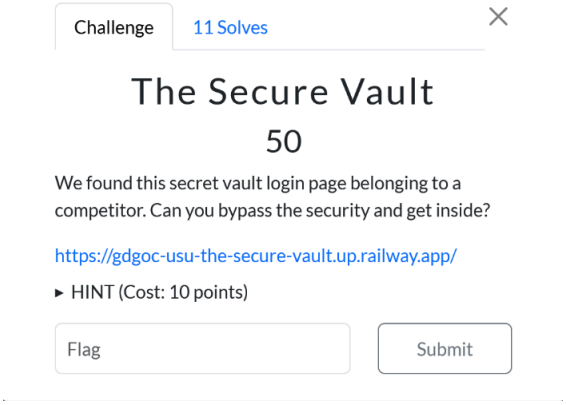
Summary

Setelah melakukan pencarian flag pada platform CTF yang telah disediakan oleh GDGoC USU untuk pengerjaan CyberSecurity Final Project, berikut adalah rangkuman hasil pengerjaan yang telah saya peroleh :

No	Challenge	Kategori	Status
1	The Secure Vault	Web Exploitation	Solved
2	Layered Secrets	Cryptography	Solved
3	The Password Checker	Reverse Engineering	Solved
4	Hidden Pixels	Forensics	Solved

Challenge 1 : The Secure Vault (Web Exploitation)

1.1 Challenge Description



Gambar 1.1 - Tampilan soal "The Secure Vault" pada CTF platform

Challenge 1 meminta peserta untuk bypass keamanan login page milik kompetitor.

1.2 Methodology & Analysis

Concepts

SQL Injection adalah jenis serangan di mana penyerang menyisipkan query SQL berbahaya ke dalam input yang dikirimkan ke aplikasi. Jika input tersebut tidak divalidasi atau di-sanitasi dengan benar oleh server, query yang disisipkan dapat tereksekusi sebagai bagian dari perintah SQL yang sah. Dampaknya bisa berupa pembacaan data sensitif, modifikasi data pada database, hingga bypass autentikasi *(Open Web Application Security Project, n.d.)*.

Vulnerabilities

Berdasarkan definisi dari **OWASP Foundation**, saya mengidentifikasi bahwa kerentanan pada challenge ini merupakan kasus umum dari SQL Injection. Pada saat melakukan *inspect* terhadap halaman login GDGoC SecureBank™, ditemukan sebuah komentar pada kode HTML yang mengindikasikan bahwa developer aplikasi tersebut menggunakan *string formatting* dalam membangun query, bukan menggunakan *parameterized queries*. Dengan pendekatan ini, perintah SQL digabungkan secara langsung dengan input pengguna tanpa proses sanitasi, sehingga ketika pengguna memasukkan **karakter khusus SQL** seperti tanda komentar (--), tanda petik (‘), atau operator logika (OR), database akan menganggap input tersebut sebagai bagian dari perintah SQL dan mengeksekusinya tanpa melakukan validasi terlebih dahulu.

Dengan adanya kerentanan ini, penyerang dapat menyisipkan payload pada halaman login GDGoC SecureBank™ untuk memanipulasi perintah yang akan dijalankan oleh database.

Berikut adalah simulasi logika dari kerentanan tersebut:

1. dibawah merupakan query dasar yang digunakan pada sistem login website

   ```
   SELECT * FROM users WHERE username = 'INPUT' AND password = 'PASSWORD'
   ```

2. Penyerang menyisipkan payload SQL Injection `admin' --` (terdapat spasi setelah tanda `--`) pada field username, dengan tujuan agar sistem membaca username sebagai admin, sementara sisa query dianggap sebagai komentar.
3. Query yang terbentuk pada database setelah input dari penyerang diproses menjadi:

   ```
   SELECT * FROM users WHERE username = 'admin' -- ' AND password = 'PASSWORD'
   ```

4. Tanda `--` menyebabkan seluruh bagian query setelahnya dianggap sebagai komentar oleh SQL, sehingga perintah yang benar-benar dieksekusi hanyalah `SELECT * FROM users WHERE username = 'admin'`, yang menyebabkan sistem berhasil melakukan autentikasi sebagai admin tanpa memerlukan password yang valid.

Logic

- Karena challenge pertama merupakan kategori Web Exploitation, pada tahap awal saya mencoba **memeriksa struktur HTML** dari halaman website tersebut terlebih dahulu, guna melihat apakah terdapat kerentanan pada keamanan halaman login serta kemungkinan adanya hint untuk mencari flag. Ketika membuka bagian Inspector, terlihat sebuah komentar pada bagian paling atas kode HTML yang berisi catatan untuk developer agar segera memperbaiki celah SQL Injection pada login query, karena query yang digunakan masih menggunakan *string formatting*, bukan *parameterized queries*.
- Berdasarkan temuan tersebut, saya mencoba melakukan **input payload** secara otomatis dengan jeda 10–15 detik antar percobaan untuk menghindari crash/down pada website, menggunakan daftar payload SQL Injection generik dari github.com/payload-box/sql-injection-payload-list melalui fitur **Repeater pada Burp Suite.**
- Namun, ketika mencoba mengakses halaman login GDGoC SecureBank™ melalui Burp Suite, server merespons dengan error **"405 Method Not Allowed – The method is not allowed for the requested URL"**. Pesan ini merupakan output standar dari framework Flask/Werkzeug, yang mengindikasikan bahwa endpoint tersebut kemungkinan hanya

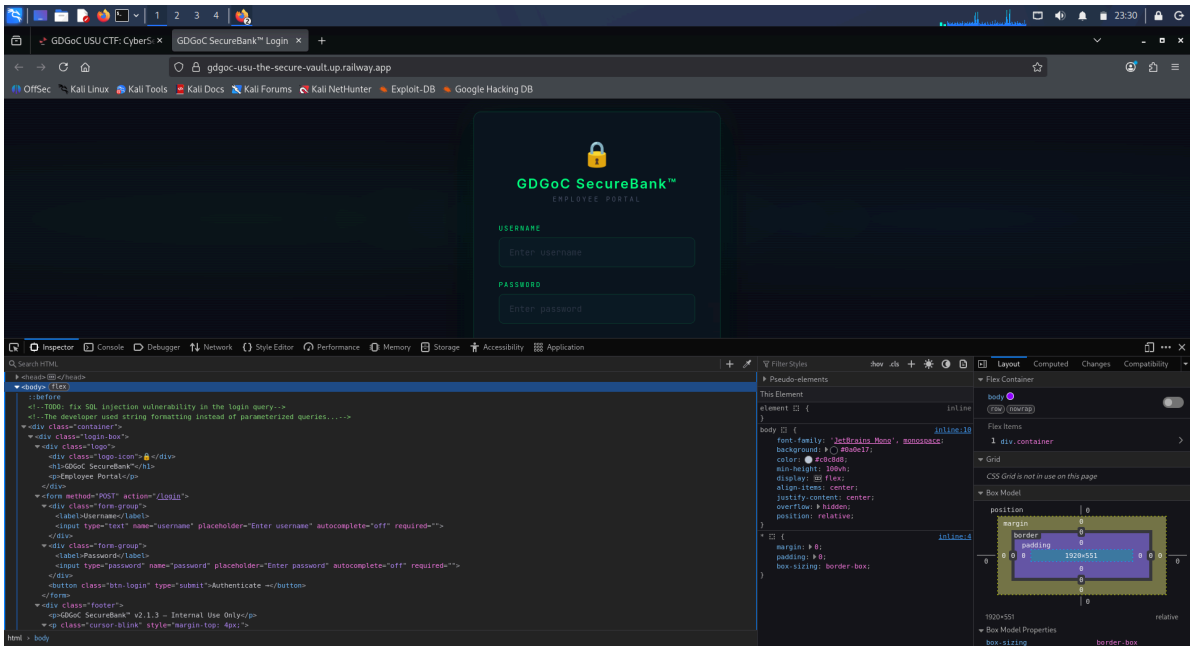
menerima HTTP method tertentu (kemungkinan besar POST), sehingga request yang dikirim dengan method yang tidak sesuai akan ditolak oleh server.

- Oleh karena itu, saya beralih melakukan input secara manual melalui form login pada browser, yang secara otomatis mengirimkan request dengan method POST yang sesuai.
- Percobaan pertama menggunakan payload SQL Injection paling sederhana dan umum digunakan, yaitu `' OR '1'='1`, dengan logika bahwa seluruh kondisi setelah `' OR '1'='1` akan selalu bernilai benar, sehingga database seharusnya tidak perlu lagi memeriksa password pengguna. Namun, **payload ini belum berhasil** melakukan bypass keamanan dan tetap menampilkan pesan "Invalid username or password". Hal ini terjadi karena berdasarkan urutan evaluasi operator SQL, AND memiliki presedensi lebih tinggi dibandingkan OR, sehingga kondisi `password = '...'` tetap harus terpenuhi meskipun kondisi `'1'='1'` sudah bernilai benar, mengingat SQL Injection hanya disisipkan pada field username.
- Selanjutnya, saya mencoba payload SQL Injection dengan menjadikan sisa query sebagai komentar, yaitu `admin' --` (dengan spasi di akhir), sehingga bagian query setelahnya dianggap tidak ada dan hanya menjalankan pencarian berdasarkan username saja. **Payload ini berhasil** melakukan bypass keamanan login. (Nilai admin pada payload dapat diganti dengan username lain secara bebas, namun dalam kasus ini digunakan username umum tersebut.)

1.3 Tools & Commands Used

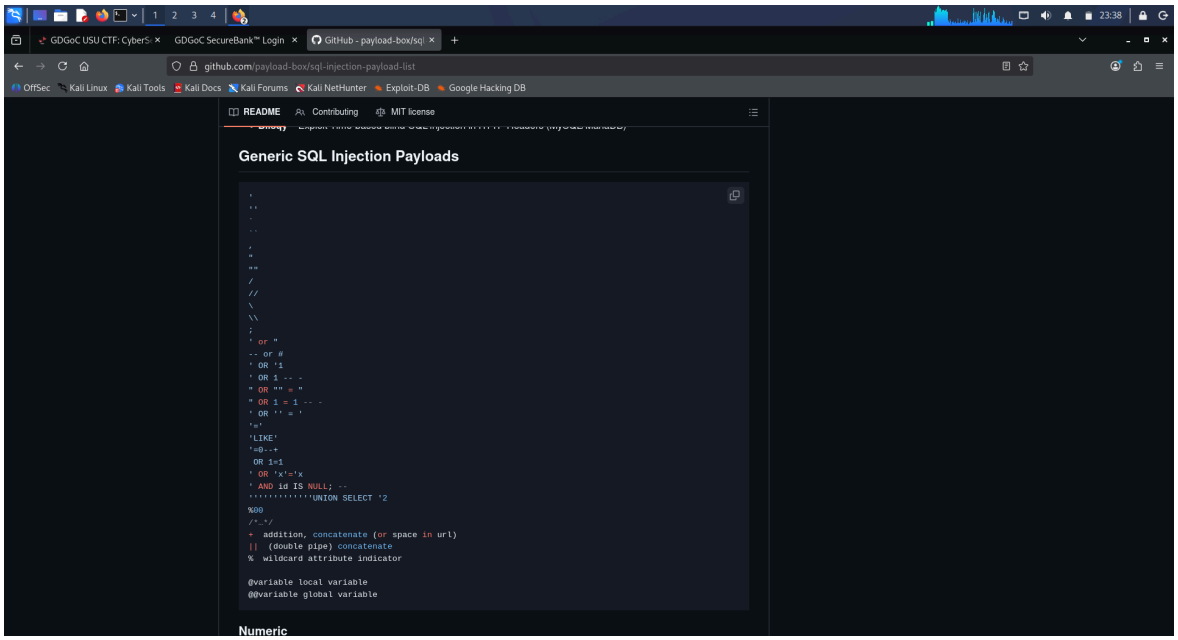
- **Browser DevTools (Inspect Element)** : untuk memeriksa struktur HTML halaman login
- **Burp Suite (Repeater)** : untuk mencoba automasi payload SQL Injection
- **Payload list** : github.com/payload-box/sql-injection-payload-list

1. Inspect HTML halaman login



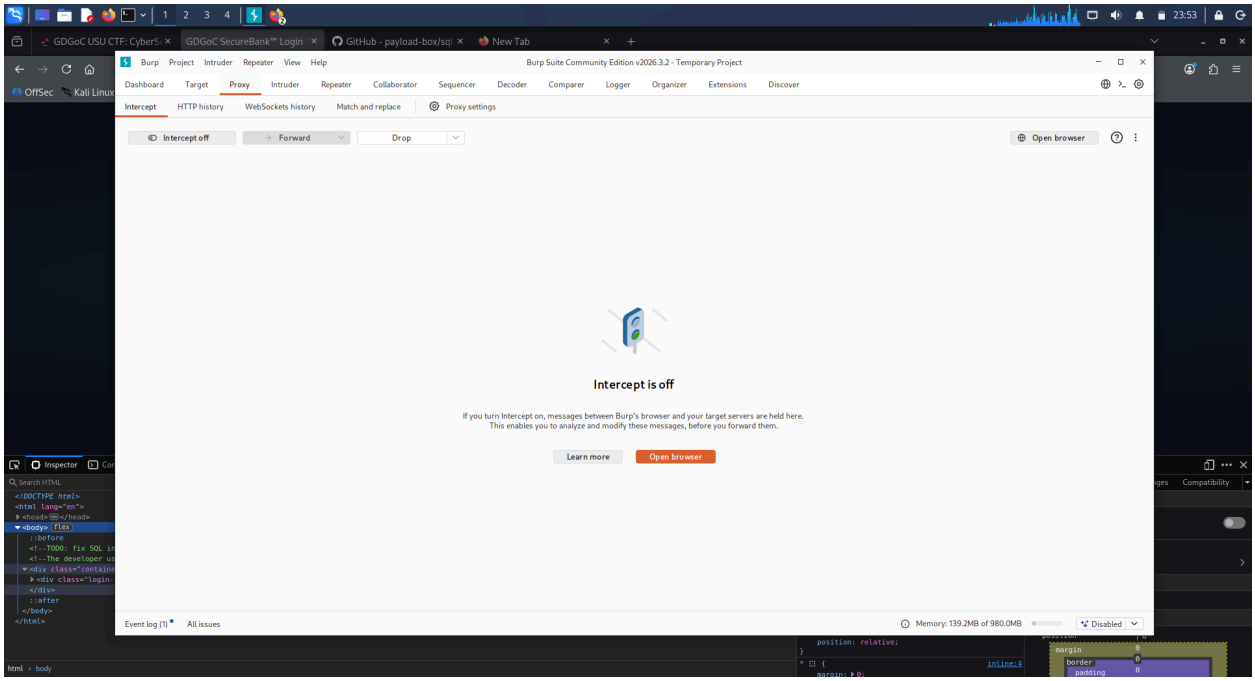
Gambar 1.3.1 - Hasil inspect

2. Generic payload SQL injection

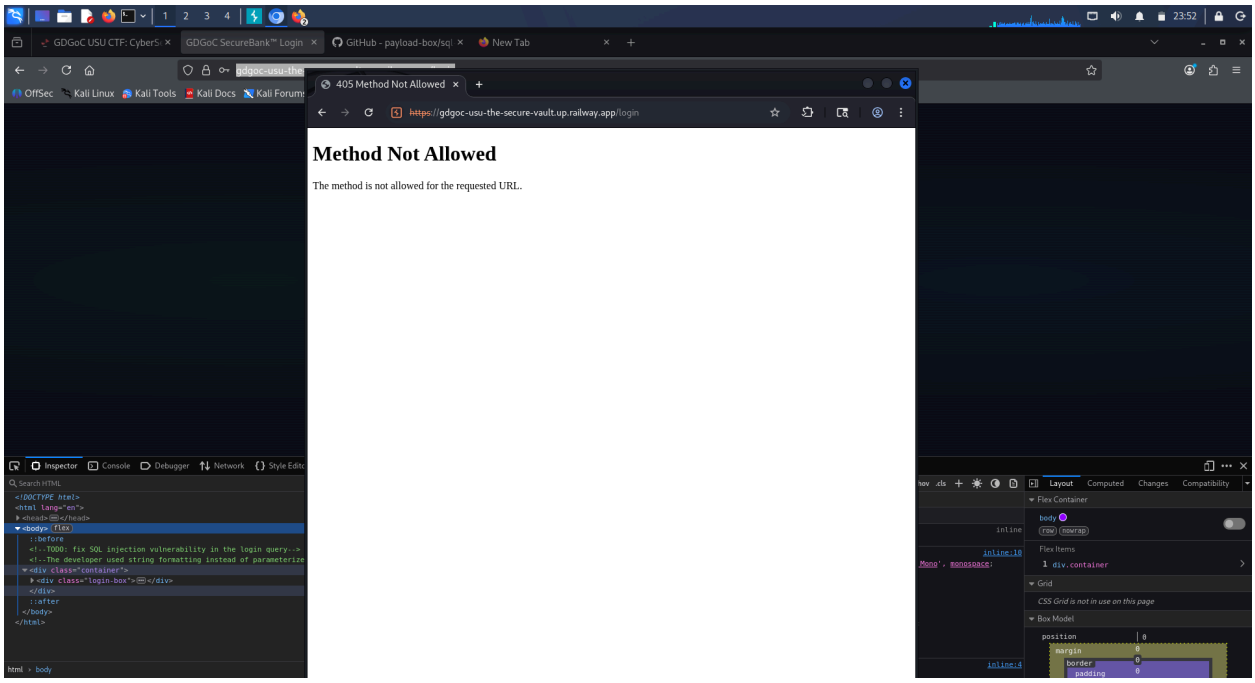


Gambar 1.3.2 - Generic payload SQL injection

3. Percobaan otomatis via Burp Suite Repeater

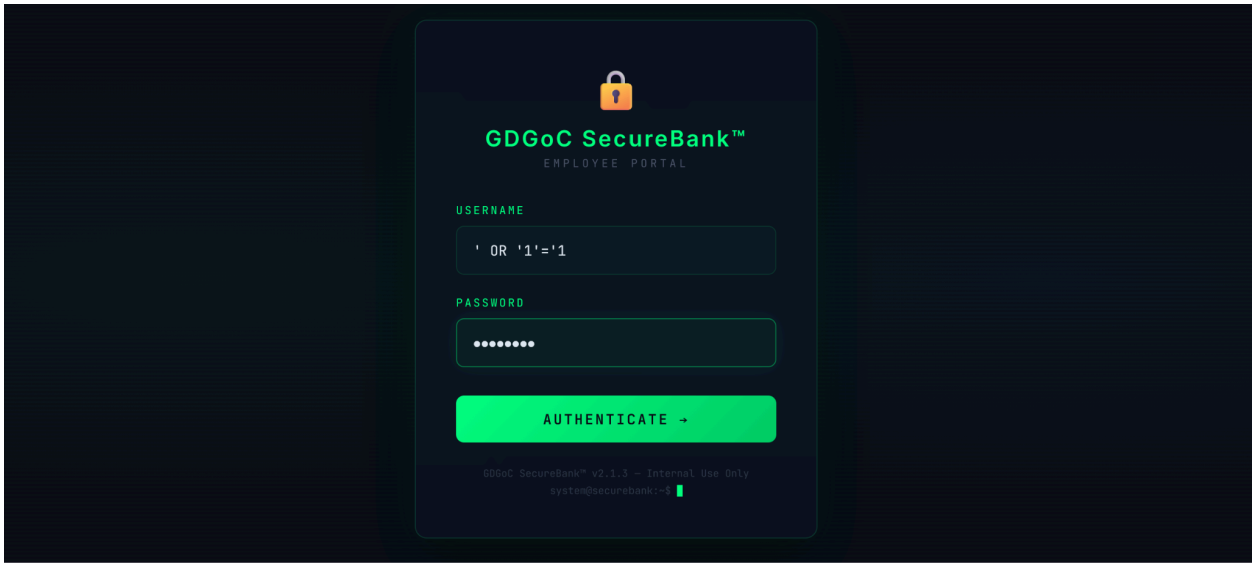


Gambar 1.3.3 - Halaman burp suite

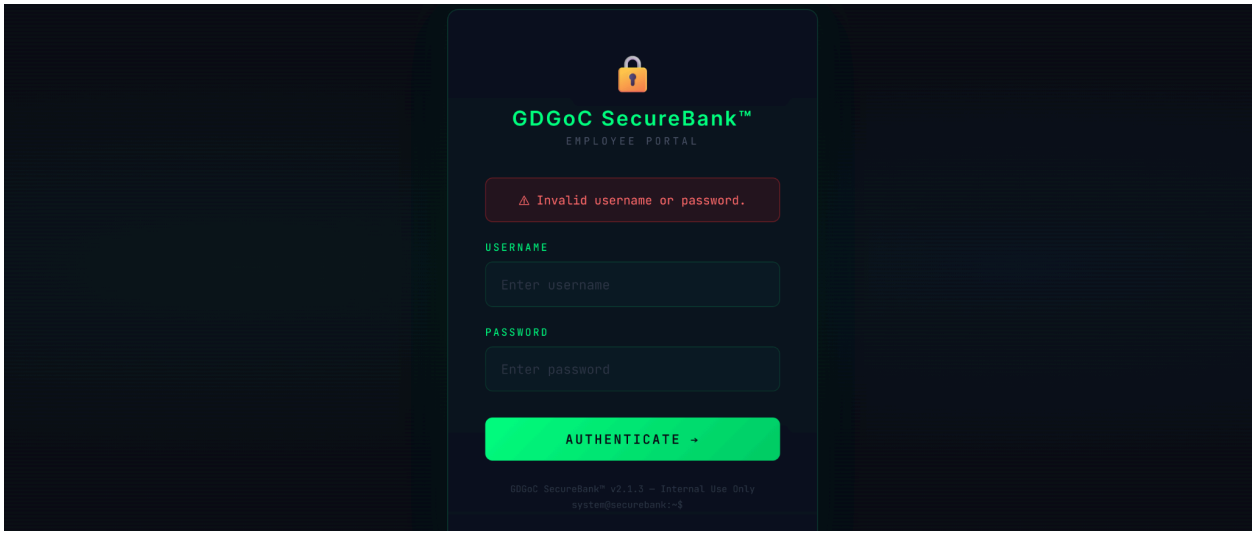


Gambar 1.3.4 - Method Not Allowed halaman login

4. Input manual : Payload gagal



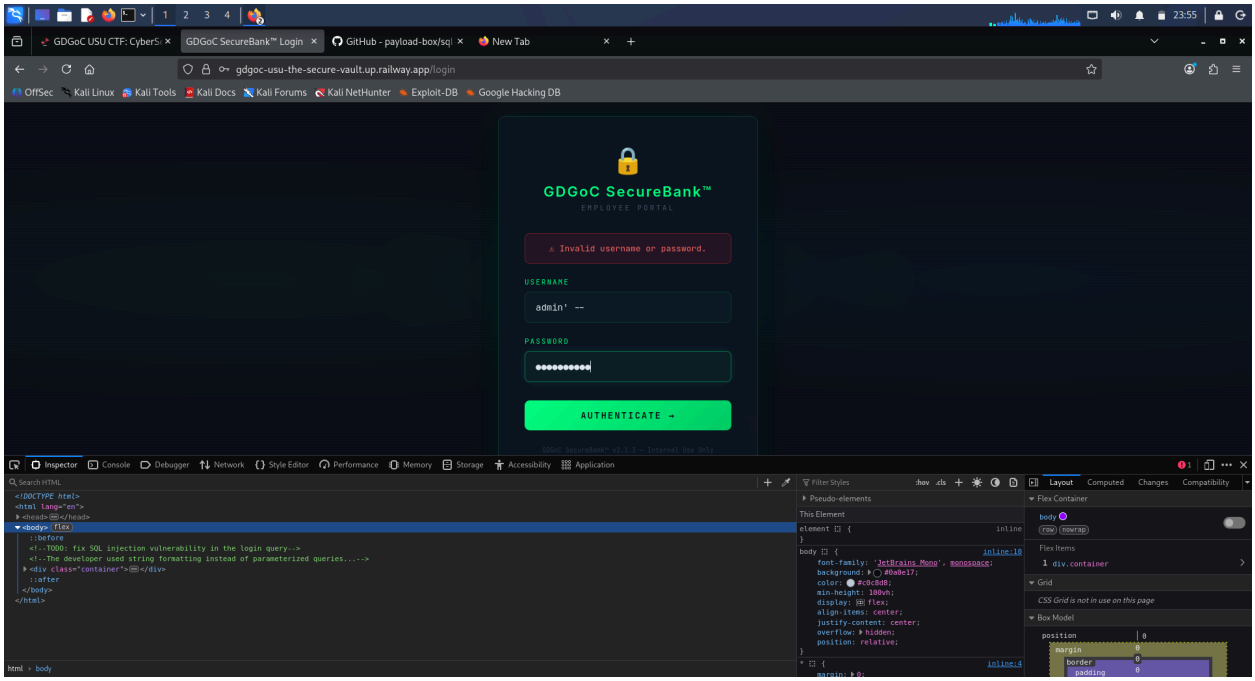
Gambar 1.3.5 - Input Payload gagal



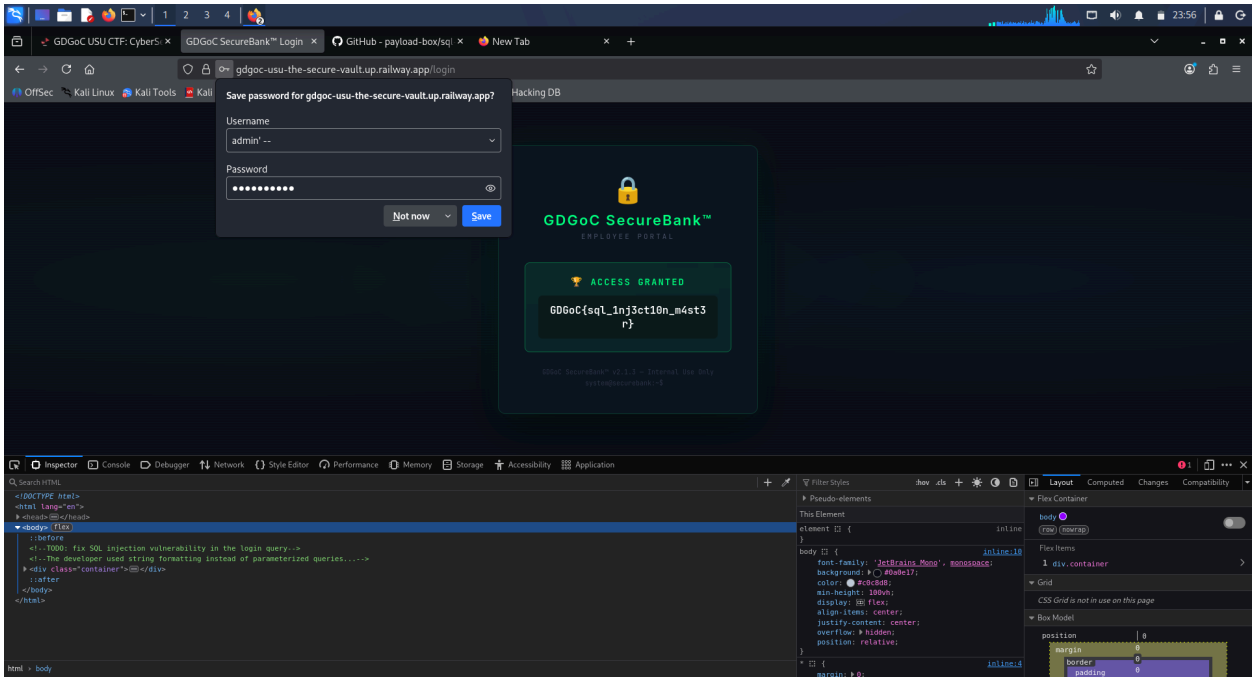
Gambar 1.3.6 - Tampilan gagal login

Field username : `` `' OR '1'='1 ``
Field password : bebas

5. Input manual : Payload berhasil



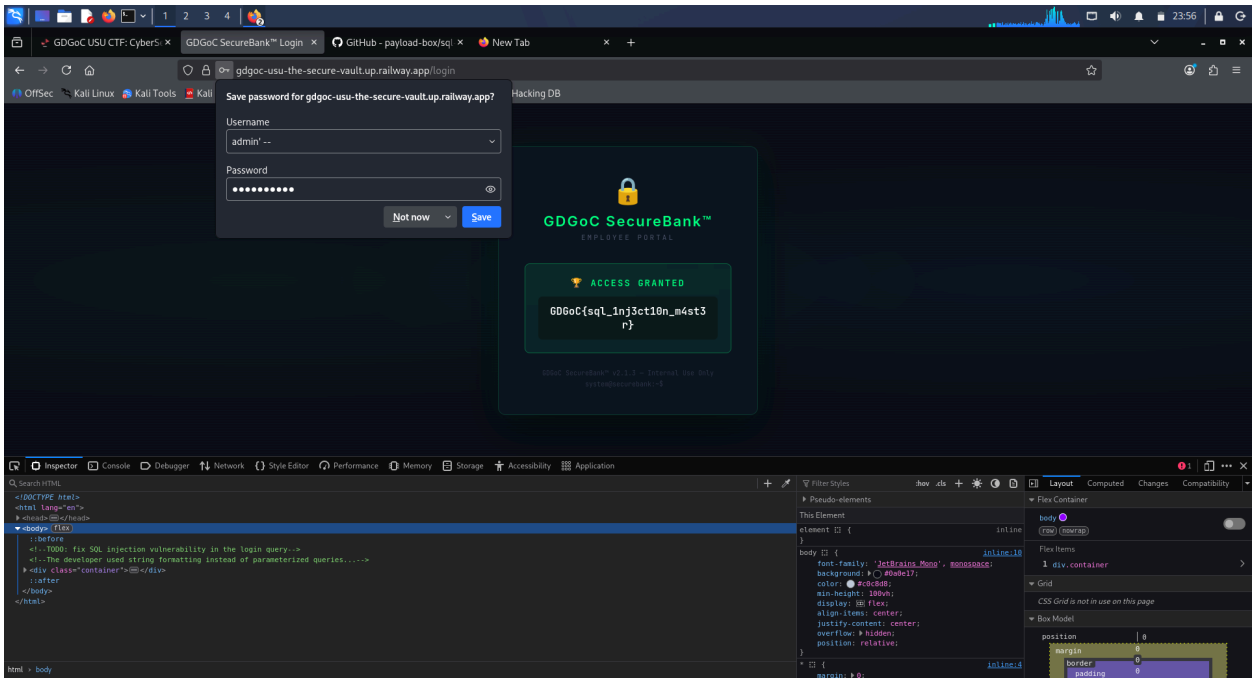
Gambar 1.3.6 - Input Payload berhasil



Gambar 1.3.7 - Tampilan berhasil login

Field username : `admin' --`
Field password : bebas

1.4 Flag



Gambar 1.4.1 - Flag challenge 1

Challenge 2 : Layered Secrets (Cryptography)

2.1 Challenge Description



Gambar 2.1 - Tampilan soal "Layered Secrets" pada CTF platform

Challenge 2 meminta peserta untuk mencari pesan rahasia yang tersembunyi dari file challenge.txt.

2.2 Methodology & Analysis

Concepts

Base64 adalah metode untuk mengubah data biner menjadi rangkaian karakter alfanumerik yang terlihat seperti teks acak. Teknik ini banyak digunakan karena dalam berbagai situasi, hanya data berbentuk teks yang dapat diterima atau ditransmisikan dengan aman, sehingga Base64 menjadi solusi untuk merepresentasikan data biner dalam bentuk teks. Namun, penggunaan Base64 memiliki konsekuensi berupa peningkatan ukuran data sekitar 33% dibandingkan data aslinya, sehingga metode ini sebaiknya hanya digunakan ketika benar-benar diperlukan *(FreeCodeCamp, n.d.)*.

Selain Base64, encoding lain yang umum digunakan adalah **Hexadecimal (hex) encoding**, yaitu representasi data dalam sistem bilangan berbasis 16, di mana setiap byte data direpresentasikan menggunakan dua digit heksadesimal (0-9 dan A-F) *(GeeksforGeeks, n.d.)*.

Selain encoding, konsep lain yang relevan pada challenge ini adalah enkripsi dan dekripsi. **Enkripsi** merupakan proses pengubahan data dari bentuk plain text menjadi ciphertext menggunakan sebuah kunci yang disebut encryption key. Sebaliknya, **dekripsi** adalah proses pengubahan ciphertext kembali menjadi plain text, yang membutuhkan kunci yang sama dengan yang digunakan pada proses enkripsi *(GeeksforGeeks, n.d.)*.

Vulnerabilities

Berdasarkan konsep encoding dan enkripsi yang telah dijelaskan sebelumnya, saya mengidentifikasi bahwa kerentanan pada challenge ini terletak pada penggunaan multi-layer encoding yang seluruhnya bersifat reversibel tanpa adanya lapisan keamanan yang kuat. Payload yang diberikan pada challenge ini di-encode secara bertingkat menggunakan **Hexadecimal encoding, Base64 encoding, dan XOR cipher** dengan key satu byte (single-byte key). Ketiga lapisan tersebut, meskipun terlihat rumit, sebenarnya dapat dibongkar tanpa kesulitan berarti karena:

- **Hex dan Base64** bukan merupakan metode enkripsi, melainkan hanya metode encoding. Keduanya tidak memerlukan kunci rahasia (key) untuk membalikkan prosesnya, sehingga siapa pun yang mengetahui bahwa data telah di-hex atau di-base64-encode dapat langsung men-decode-nya tanpa hambatan.
- **XOR cipher** dengan key satu byte hanya memiliki 256 kemungkinan kombinasi key (0x00 hingga 0xFF), sehingga rentan terhadap **brute-force attack**. Dengan mencoba

seluruh 256 kemungkinan key secara otomatis, penyerang dapat dengan mudah menemukan key yang tepat, terlebih apabila format hasil akhir (plaintext) sudah diketahui. dalam kasus ini, format flag `GDGoC{...}`, sehingga proses pencarian key dapat divalidasi secara otomatis oleh program.

Dengan adanya kerentanan ini, penyerang dapat melakukan reverse-engineering terhadap seluruh proses encoding secara berurutan dari lapisan terluar hingga lapisan terdalam untuk mengungkap flag asli. Simulasi logika dari proses tersebut adalah sebagai berikut.

1. Payload awal yang diberikan pada challenge berbentuk rangkaian karakter heksadesimal

   `425159464c514535495441374d6a5a794853417763585970635441 2f`

2. Payload tersebut di-decode dari hex menjadi teks, menghasilkan string yang menyerupai format Base64

   `BQYFLQE5ITA7MjZyHSAwcXYpcTA/`

3. String tersebut kemudian di-decode dari Base64, menghasilkan rangkaian byte yang tidak terbaca (non-printable), yang mengindikasikan adanya **satu lapisan enkripsi tambahan berupa XOR cipher.**
4. Dilakukan proses brute-force terhadap seluruh 256 kemungkinan key XOR satu byte, dengan mencocokkan hasil decode terhadap format flag yang diketahui (`GDGoC{...}`). Pada key 0x42, ditemukan hasil decode yang valid, yaitu:

   `GDGoC{crypt0_br34k3r}`

Logic

- Pertama-tama, saya mencoba **mengidentifikasi jenis encoding** yang digunakan dengan bantuan situs caesarcipher.org, sebuah tools online yang dapat mendeteksi kemungkinan jenis enkripsi/encoding dari suatu string, meskipun hasilnya tidak selalu 100% akurat. Tools ini saya gunakan sebagai referensi awal untuk menentukan operasi apa yang perlu diterapkan pada CyberChef, sekaligus untuk **mengonfirmasi** hasil decode di setiap tahap. Berdasarkan analisis struktur payload yang didominasi oleh karakter angka dan huruf a-f, saya menduga bahwa payload tersebut merupakan hasil encoding Hexadecimal, sehingga saya menerapkan operasi From Hex pada **CyberChef.**
- Setelah mendapatkan hasil decode dari tahap pertama, saya kembali menggunakan caesarcipher.org untuk mengidentifikasi jenis encoding pada hasil tersebut. Berdasarkan hasil identifikasi serta ciri khas berupa keberadaan tanda garis miring (`/`) pada bagian akhir string yang merupakan **karakter umum dalam alfabet Base64**. saya menduga

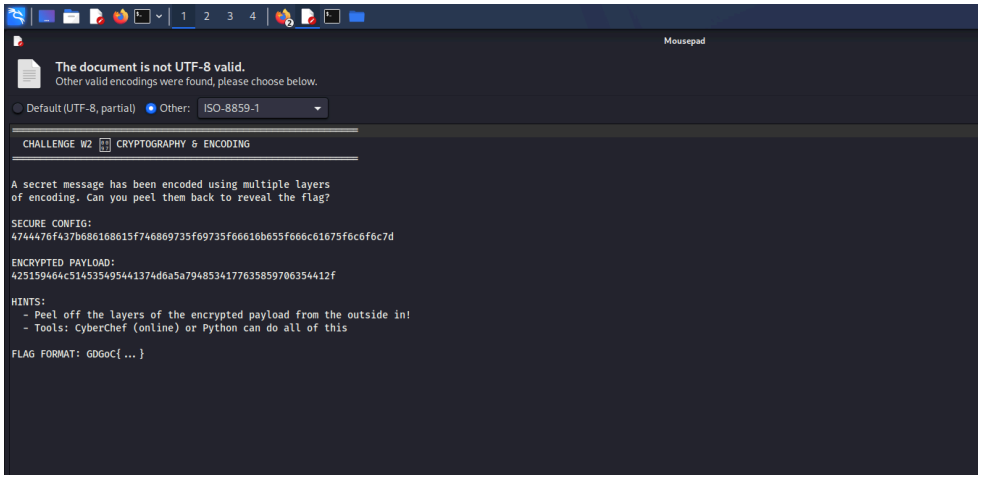
hasil tersebut merupakan encoding Base64, sehingga saya menerapkan operasi From Base64 pada CyberChef.

- Setelah operasi From Base64 diterapkan, hasil yang didapatkan berupa rangkaian byte yang bersifat non-printable (tidak dapat dibaca sebagai teks) dan tidak lagi dapat diidentifikasi jenis encoding-nya melalui caesarcipher.org. Hal ini mengindikasikan bahwa data tersebut bukan lagi berupa encoding biasa, melainkan telah melalui proses enkripsi, kemungkinan besar menggunakan XOR cipher, **mengingat XOR** merupakan salah satu teknik enkripsi paling umum digunakan dalam CTF karena sifatnya yang ringan namun tetap membuat data menjadi tidak terbaca tanpa mengetahui key yang digunakan.
- Karena XOR cipher pada umumnya menggunakan key satu byte (karena tidak ada indikasi penggunaan key dengan panjang tertentu), saya menyadari bahwa jumlah kemungkinan key hanya sebanyak 256 (0x00 hingga 0xFF). Dengan mengetahui format flag yang harus dihasilkan (`GDGoC{...}`), saya melakukan brute-force terhadap seluruh **256 kemungkinan** key tersebut menggunakan **CyberChef dan script Python**, dengan mencocokkan setiap hasil decode terhadap format flag yang diketahui. Proses ini berhasil **menemukan key = 0x42,** yang menghasilkan flag yang valid, yaitu `GDGoC{crypt0_br34k3r}`.

2.3 Tools & Commands Used

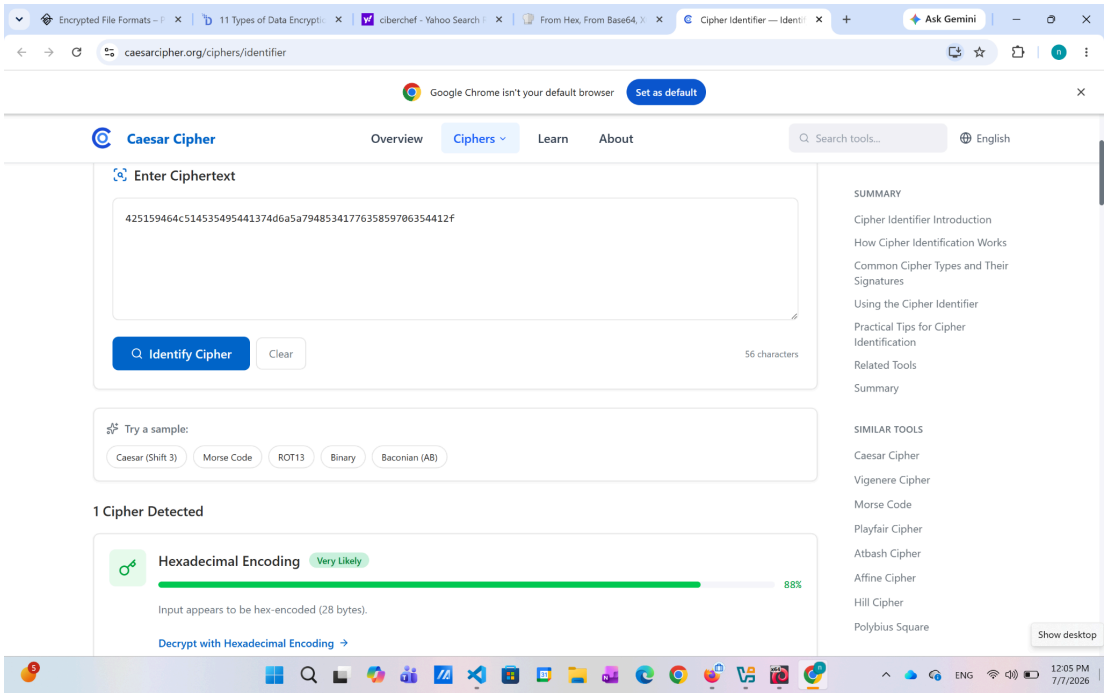
- **caesarcipher.org** : untuk mengidentifikasi kemungkinan jenis encoding/enkripsi pada tiap tahap
- **CyberChef** : untuk melakukan proses decode (From Hex, From Base64) dan brute-force XOR key
- **Visual Studio Code (VSCode)** : untuk menulis dan menjalankan solver script Python

1. Isi file dari challenge.txt setelah di download



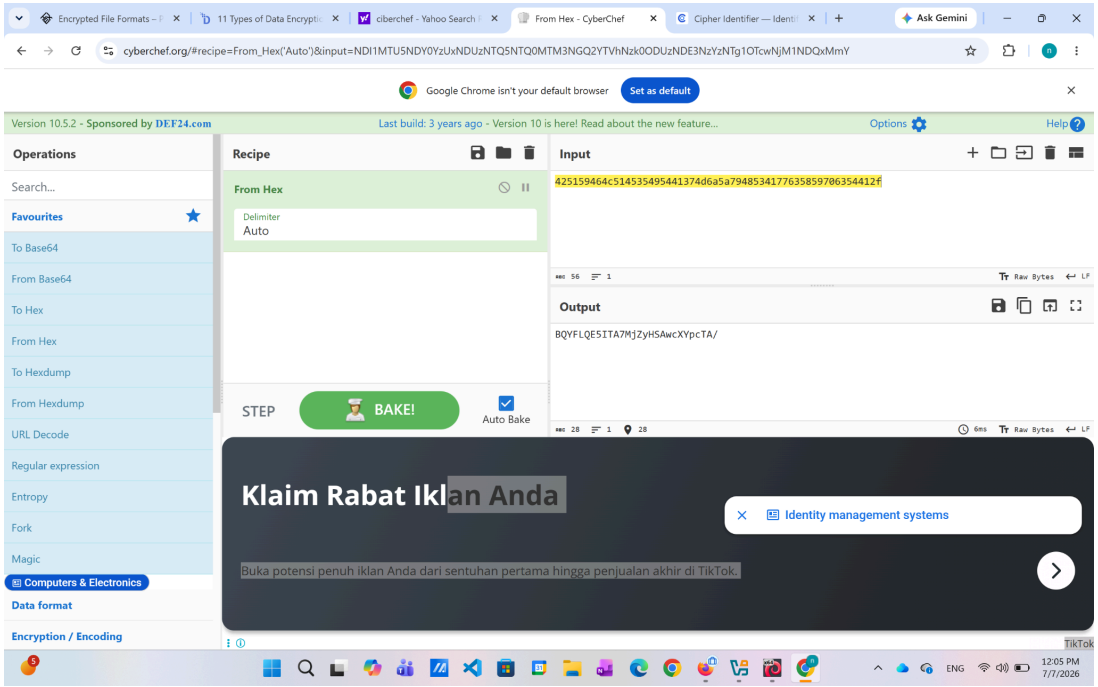
Gambar 2.3.1 - Tampilan challenge.txt

2. Identifikasi payload awal menggunakan caesarcipher.org



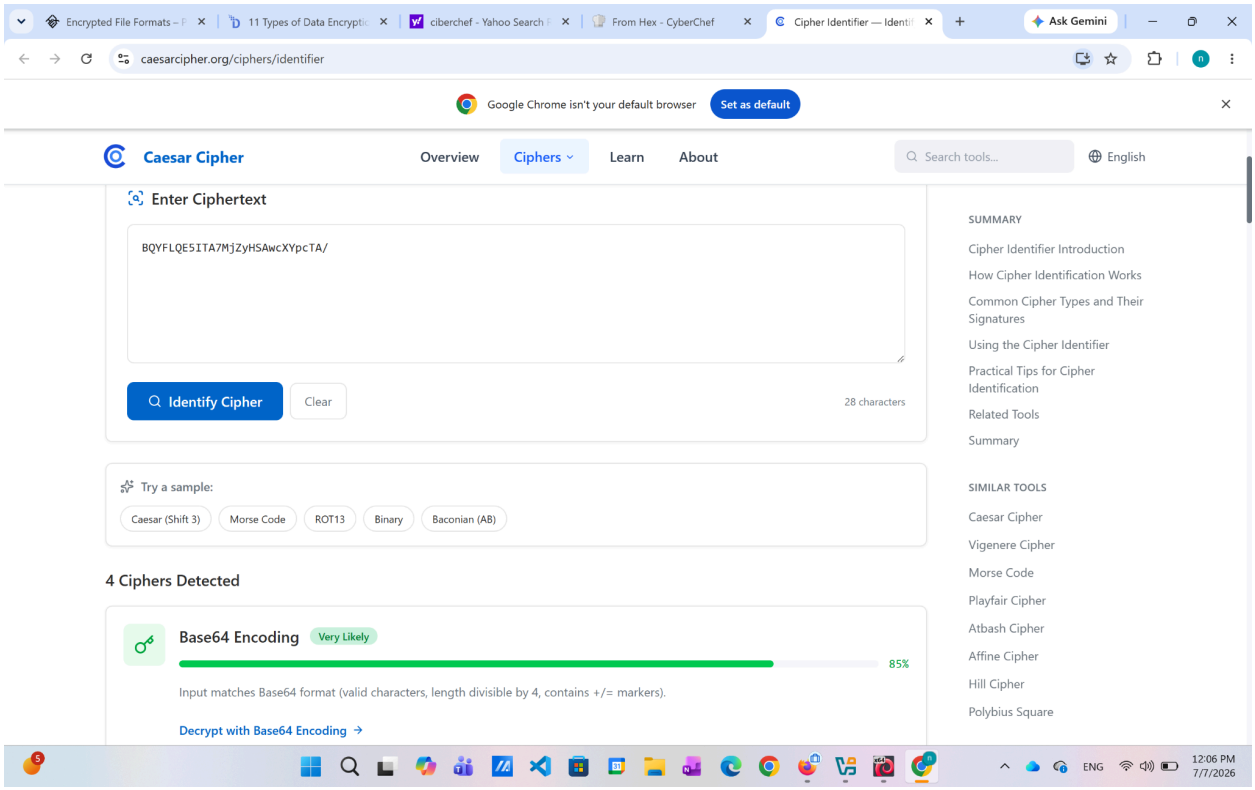
Gambar 2.3.2 - Hasil identifikasi payload awal pada caesarcipher.org

3. Menerapkan operasi From Hex pada CyberChef



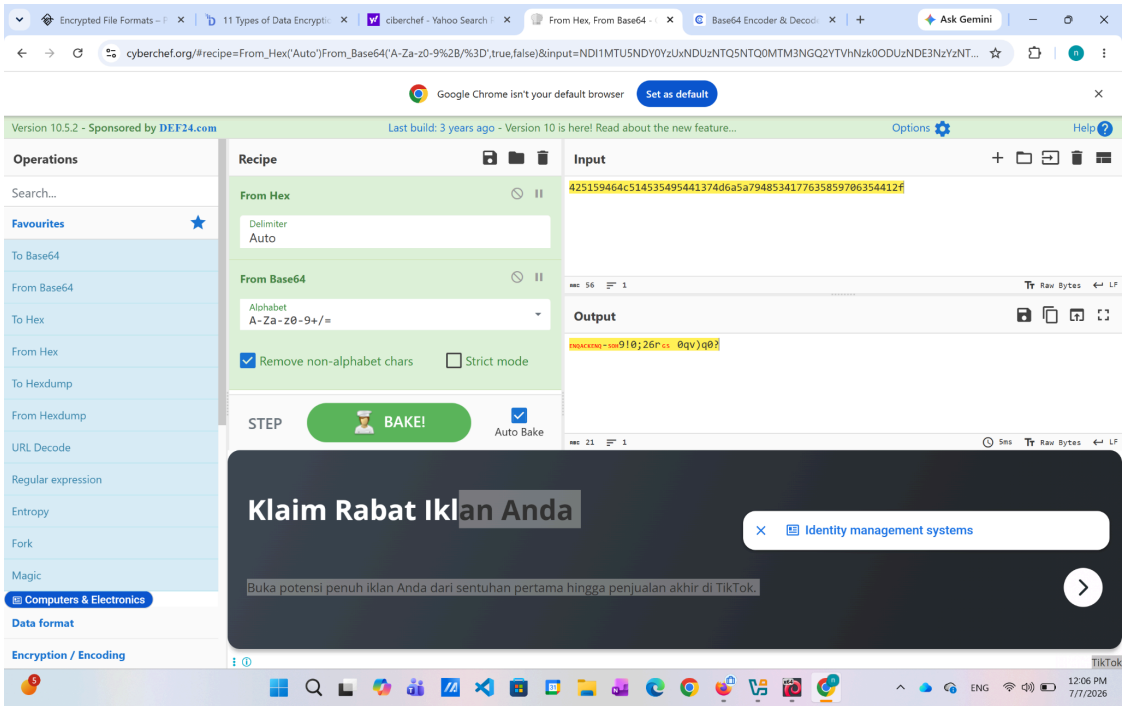
Gambar 2.3.3 - Hasil decode From Hex pada CyberChef

4. Identifikasi hasil decode hex menggunakan caesarcipher.org



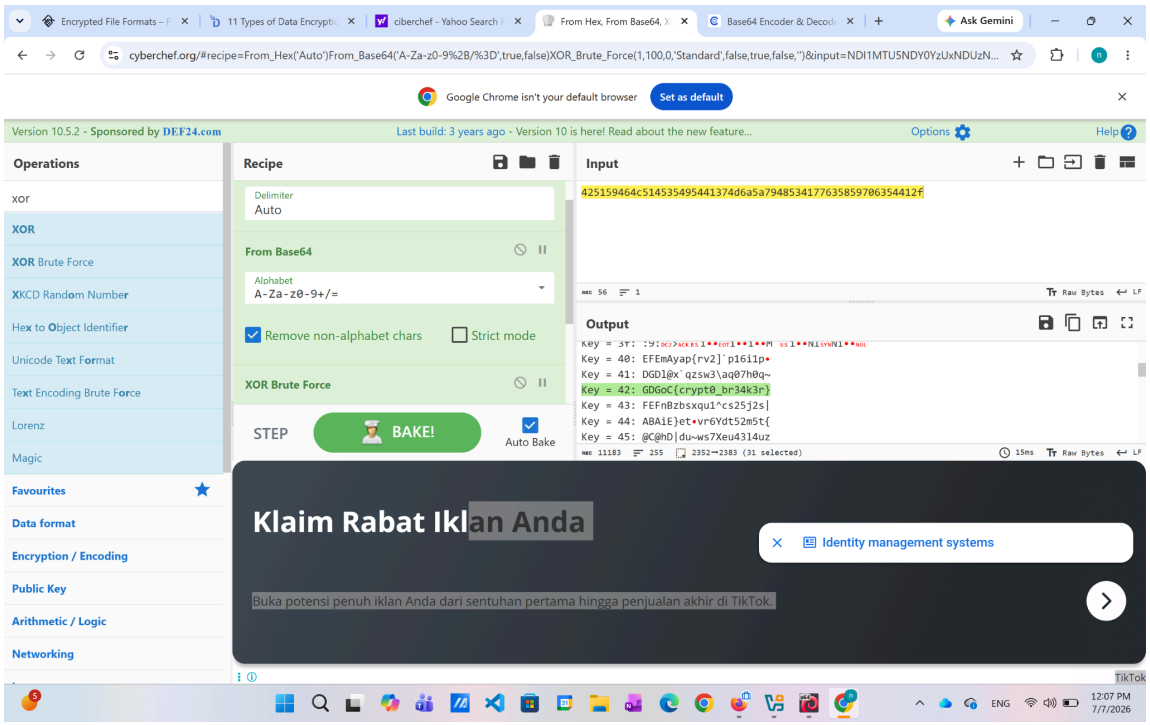
Gambar 2.3.4 - Hasil identifikasi encoding Base64 pada caesarcipher.org

5. Menerapkan operasi From Base64 pada CyberChef



Gambar 2.3.5 - Hasil decode From Base64 pada CyberChef (bytes non-printable

6. Melakukan brute-force XOR key pada CyberChef



Gambar 2.3.6 - Hasil XOR Brute Force pada CyberChef, menampilkan key 0x42

2.4 Solver Script

```python
"""
GDGoC CTF - Cryptography Challenge Solver
Layers: Hex -> Base64 -> XOR (single-byte key)

file name:
    python3 solve_crypto.py
"""
import base64

# Payload asli dari soal, masih dalam bentuk hex string
ENCRYPTED_PAYLOAD = "425159464c514535495441374d6a5a7948534177635859706354412f"


def solve():
    # Layer 1: payload di-decode dari hex ke bytes pakai bytes.fromhex()
    step1 = bytes.fromhex(ENCRYPTED_PAYLOAD)
    print("[Layer 1] Hex decoded ->", step1.decode())

    # Layer 2: hasil dari layer 1 ternyata berbentuk base64, jadi di-decode lagi
    step2 = base64.b64decode(step1)
    print("[Layer 2] Base64 decoded ->", step2)

    # Layer 3: hasil layer 2 berupa bytes yang gak kebaca (non-printable),
    # jadi kemungkinan ini XOR cipher. Karena cuma 1 byte key, kemungkinannya
    # cuma 256 (0-255), jadi bisa di-brute-force semua sekaligus
    print("[Layer 3] Brute-forcing XOR key...")
    for key in range(256):
        # Coba XOR tiap byte di step2 pakai key yang lagi di-loop
        candidate = bytes([b ^ key for b in step2])
        try:
            text = candidate.decode()
        except UnicodeDecodeError:
            # kalau hasil XOR-nya gak valid jadi text, skip aja, lanjut key berikutnya
            continue

        # cek apakah hasilnya cocok sama format flag yang diminta soal
        if text.startswith("GDGoC{") and text.endswith("}"):
            print(f"\nFound key: 0x{key:02x} ({key})")
            print(f"FLAG: {text}")
            return text

    print("No matching key found.")
    return None


if __name__ == "__main__":
    solve()
```

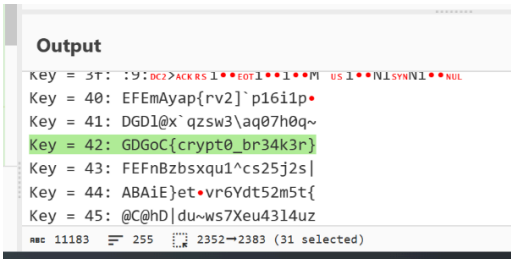
```
PS C:\Growth\python 2> python -u "c:\Growth\python 2\tempCodeRunnerFile.py"
[Layer 1] Hex decoded -> BQYFLQE5ITA7MjZyHSAwcXYpcTA/
[Layer 2] Base64 decoded -> b'\x05\x06\x05-\x019!0;26r\x1d 0qv)q0?'
[Layer 3] Brute-forcing XOR key...

Found key: 0x42 (66)
FLAG: GDGoC{crypt0_br34k3r}
PS C:\Growth\python 2>
```

Gambar 2.4.1 - Script Python solve_crypto

Command : `python solve_crypto.py`

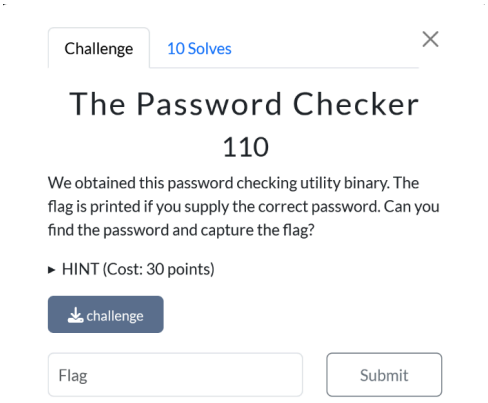
2.5 Flag



Gambar 2.5.1 - Flag challenge 2

Challenge 3 : The Password Checker (Reverse Engineering)

3.1 Challenge Description



Gambar 3.1 - Tampilan soal "The Password Checker" pada CTF platform

Challenge 3 meminta peserta untuk mencari password dari file challenge.

3.2 Methodology & Analysis

Concepts

Reverse engineering adalah proses menganalisis suatu perangkat lunak untuk memahami cara kerja internalnya tanpa memiliki akses terhadap source code asli, biasanya dilakukan dengan cara membongkar (disassemble) atau melakukan dekompilasi (decompile) terhadap file binary *(GeeksforGeeks, n.d.)*. Salah satu tools yang umum digunakan untuk melakukan analisis statis (static analysis) terhadap binary adalah Ghidra, sebuah software reverse engineering yang dikembangkan oleh NSA (National Security Agency) dan bersifat open-source, yang memungkinkan pengguna untuk melihat representasi kode assembly maupun pseudo-code (mendekati bahasa C) dari suatu binary *(Ghidra-SRE, n.d.)*.

Pada challenge ini, teknik yang digunakan untuk menyembunyikan flag di dalam binary adalah **XOR cipher**, yaitu teknik enkripsi sederhana yang mengoperasikan setiap bit data dengan sebuah kunci menggunakan operator XOR (exclusive OR), di mana proses dekripsinya dapat dilakukan dengan mengulangi operasi XOR yang sama menggunakan kunci yang sama *(Wikipedia, n.d.)*.

Vulnerabilities

Berdasarkan konsep reverse engineering dan XOR cipher yang telah dijelaskan sebelumnya, saya mengidentifikasi bahwa kerentanan pada challenge ini **terletak pada penyimpanan data sensitif** (password dan flag) dalam bentuk ter-obfuscate menggunakan XOR cipher dengan key statis, yang tersimpan secara langsung di dalam binary. Pada saat melakukan analisis terhadap hasil dekompilasi fungsi `main()` menggunakan Ghidra, ditemukan dua buah array byte yang di-hardcode langsung dalam kode, yaitu `local_97` (14 byte) dan `local_b8` (23 byte). Kedua array tersebut tidak merepresentasikan teks yang dapat dibaca secara langsung, yang mengindikasikan bahwa data di dalamnya telah melalui proses enkripsi sederhana sebelum program di-compile.

Dengan menelusuri alur logika program lebih lanjut, ditemukan bahwa program menggunakan operasi XOR pada dua bagian kode yang berbeda:

- Pada baris `(local_97[(long)local_18 + 0xf] ^ 0x39) != (uint)local_97[local_18]`, program membandingkan input password yang dimasukkan pengguna (setelah di-XOR dengan key 0x39) terhadap array `local_97` yang telah di-hardcode. Baris ini merupakan mekanisme **validasi password**, di mana password yang dimasukkan pengguna dianggap benar apabila hasil XOR-nya sama persis dengan array target.
- Pada baris `local_d8[local_1c] = local_b8[local_1c] ^ 0x5a`, yang hanya dieksekusi apabila validasi password pada poin sebelumnya berhasil, program

melakukan XOR terhadap array `local_b8` menggunakan key 0x5a untuk menghasilkan flag asli, yang kemudian langsung ditampilkan melalui `printf("[+] Flag: %s\n", local_d8)`.

Dengan adanya kerentanan ini, **penyerang dapat melakukan reverse-engineering** terhadap kedua operasi XOR tersebut untuk memperoleh password yang valid maupun flag secara langsung, tanpa perlu menjalankan binary maupun melakukan brute-force terhadap password. Simulasi logika dari proses tersebut adalah :

1. Array `local_97` yang digunakan untuk validasi password (byte 0 hingga 0xd) di-XOR menggunakan key 0x39, menghasilkan **password yang valid**

   `gdgoc_2024_ctf`

2. Array `local_b8` yang digunakan untuk menghasilkan flag (byte 0 hingga 0x16) di-XOR menggunakan key 0x5a, **menghasilkan flag asli secara langsung**

   `GDGoC{r3v3rs3_3ng1n33r}`

3. Dengan demikian, flag dapat diperoleh tanpa perlu menjalankan binary sama sekali, cukup dengan melakukan analisis statis terhadap hasil dekompilasi dan menghitung ulang operasi XOR secara manual maupun menggunakan script.

Yang dicari	Baris Pseudo Code	Key XOR
Password check	`(local_97[local_18 + 0xf] ^ 0x39) != (uint)local_97[local_18]`	0x39
Flag generator	`local_d8[local_1c] = local_b8[local_1c] ^ 0x5a`	0x5a

Logic

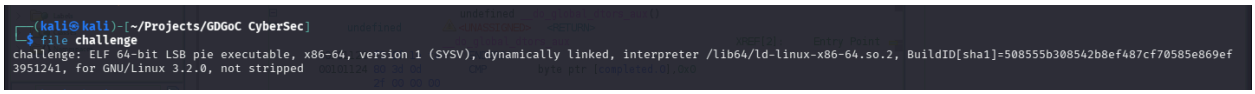
- Setelah mengunduh file challenge, saya melakukan pengecekan awal menggunakan perintah `file challenge` pada terminal Kali Linux untuk mengetahui tipe file tersebut. Hasilnya menunjukkan bahwa file merupakan **ELF 64-bit LSB** pie executable, x86-64, yaitu format file executable pada sistem operasi Linux, sehingga file dapat langsung dijalankan maupun dianalisis lebih lanjut.

- Sebelum melakukan analisis lebih mendalam menggunakan Ghidra, saya melakukan pengecekan sederhana menggunakan perintah `strings challenge | grep -i "GDGoC"` untuk mencari kemungkinan adanya string flag yang tertanam secara langsung (plaintext) di dalam binary. Hasil pencarian menemukan sebuah string dengan format flag, namun setelah dianalisis lebih lanjut, string tersebut merupakan fake flag yang sengaja disisipkan sebagai jebakan (decoy).
- Selanjutnya, file dibuka menggunakan Ghidra untuk dilakukan proses import dan analisis otomatis (auto-analysis), kemudian saya membuka fungsi `main()` melalui panel Symbol Tree untuk melihat hasil dekompilasi dalam bentuk pseudo-code. Pada tahap ini, ditemukan keanehan berupa dua variabel array (`local_97 dan local_b8`) yang diisi menggunakan rangkaian nilai byte dalam format heksadesimal satu per satu, bukan berupa string literal yang dapat langsung dibaca. Hal ini mengindikasikan bahwa kedua array tersebut merupakan data yang telah di-obfuscate, bukan data teks biasa.
- Dengan menelusuri alur logika program lebih lanjut, **ditemukan bahwa kedua array tersebut diproses** menggunakan operator `XOR (^)` pada dua bagian kode yang berbeda: satu digunakan untuk melakukan validasi password terhadap input pengguna, dan satu lagi digunakan untuk menghasilkan flag setelah validasi berhasil. Untuk membuktikan hal ini, saya mencoba melakukan perhitungan manual dengan mengonversi setiap nilai byte ke dalam bentuk biner, kemudian melakukan operasi XOR terhadap key yang ditemukan pada kode (0x39 untuk password dan 0x5a untuk flag), dan mengonversi hasilnya ke karakter ASCII menggunakan **kalkulator serta tabel ASCII**.
- Karena proses perhitungan manual untuk seluruh byte (14 byte untuk password dan 23 byte untuk flag) cukup memakan waktu dan rawan kesalahan, saya menulis **script Python sederhana** untuk mengotomatisasi proses XOR tersebut, sehingga password dan flag dapat diperoleh secara langsung dan lebih akurat.

3.3 Tools & Commands Used

- **Ghidra :** untuk melakukan dekompilasi dan analisis statis terhadap binary
- **Visual Studio Code (VSCode)** : untuk menulis dan menjalankan solver script Python
- **Kali Linux Terminal** : untuk menjalankan command-line tools

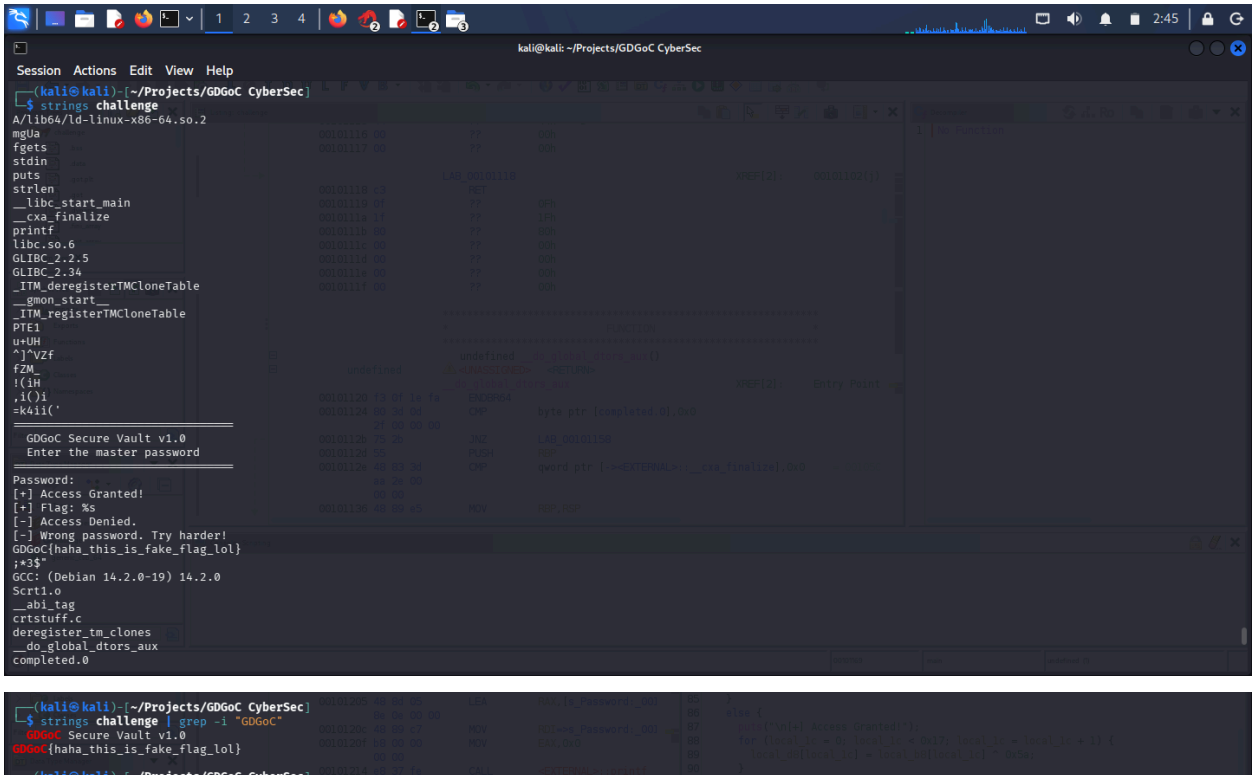
1. Cek tipe file menggunakan file



Gambar 3.3.1 - Hasil pengecekan tipe file menggunakan command file challenge

Command : `file challenge`

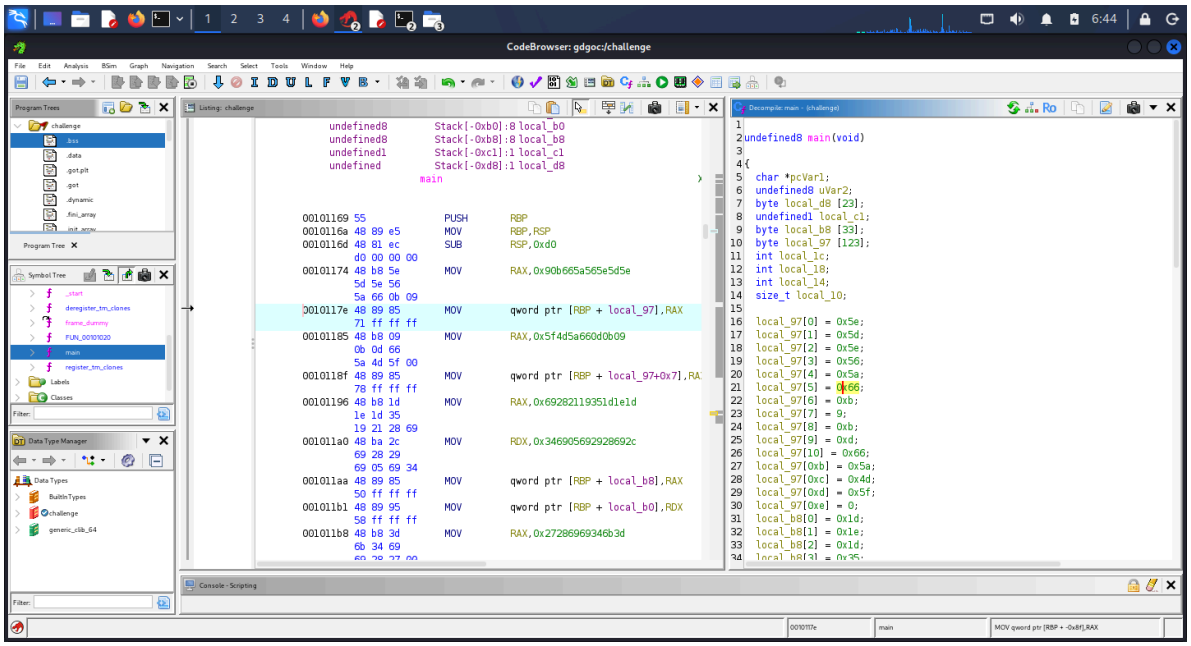
2. Cek string flag menggunakan strings dan grep



Gambar 3.3.2 - Hasil pencarian string pada binary, ditemukan fake flag

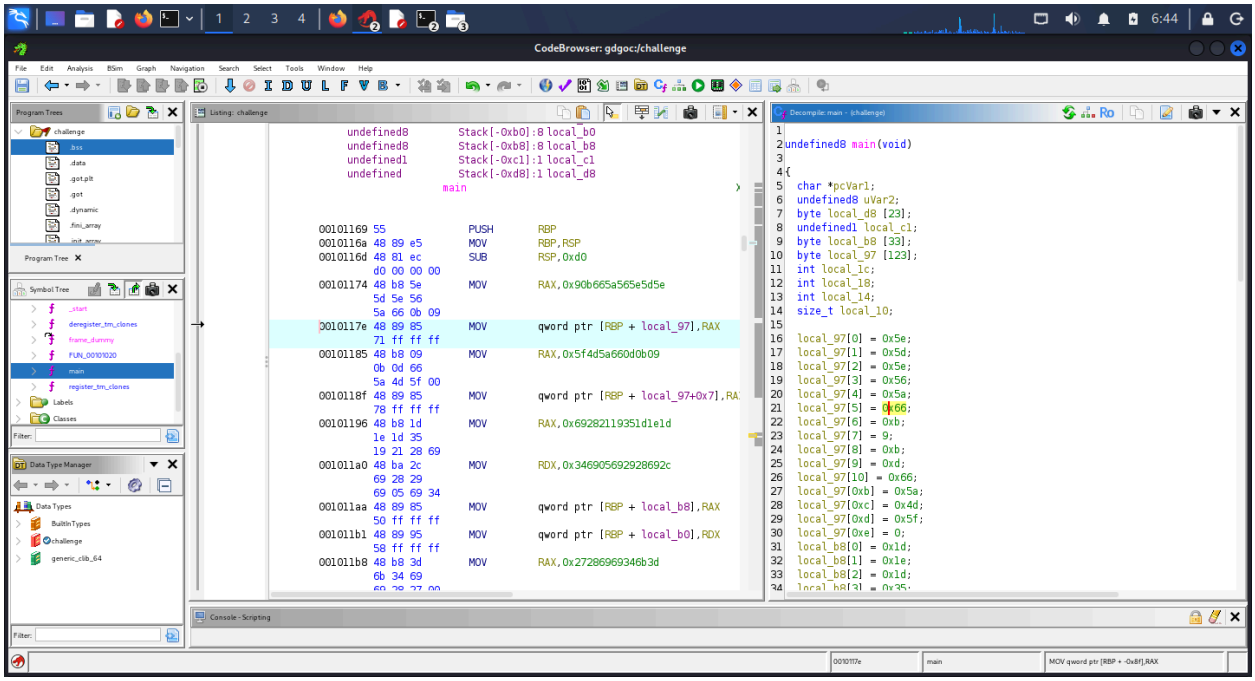
Command : `strings challenge , strings challenge | grep -i "GDGoC"`

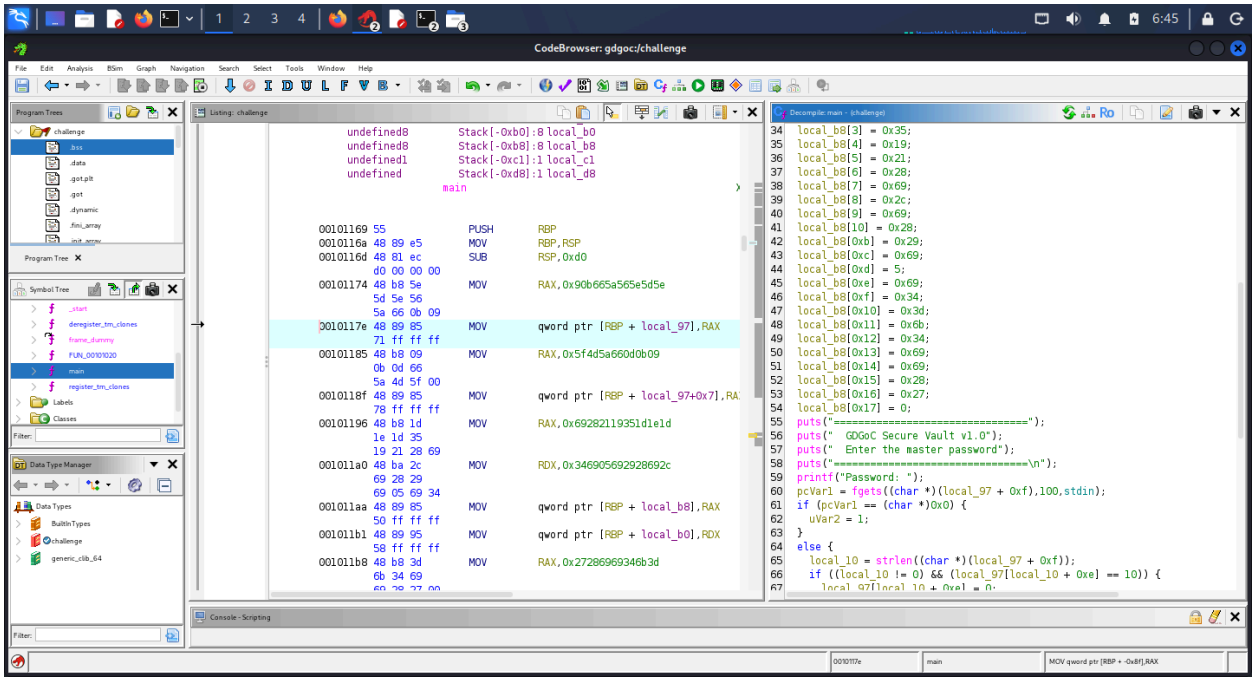
3. Import dan analisis binary pada Ghidra

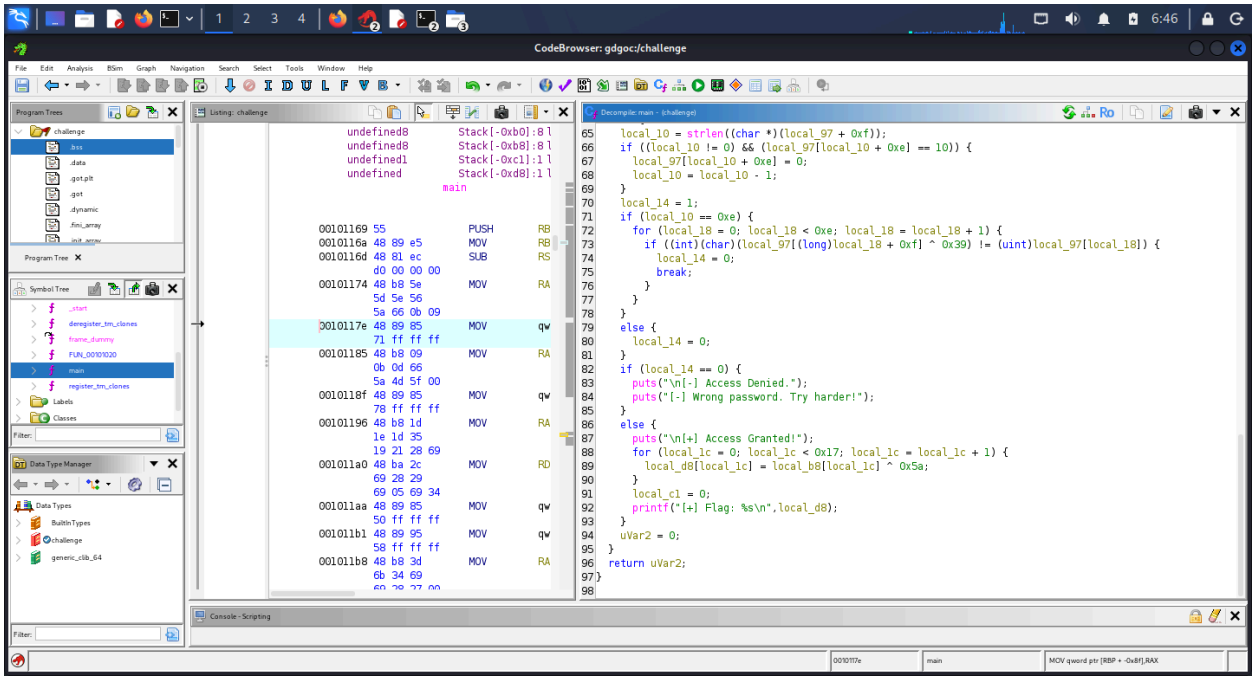


Gambar 3.3.3 - Tampilan CodeBrowser setelah proses auto-analysis pada Ghidra

4. Hasil dekompilasi fungsi main()







Gambar 3.3.4 - Pseudo-code hasil dekompilasi fungsi main() pada Ghidra, menampilkan array local_97 dan local_b8

5. Menjalankan solver script menggunakan VSCode

```
PS C:\Growth\python 2> python -u "c:\Growth\python 2\solve_reverse.py"
Recovered password: gdgoc_2024_ctf
Recovered flag:     GDGoC{r3v3rs3_3ng1n33r}
PS C:\Growth\python 2>
```

Gambar 3.3.5 - Solver script pada VSCode beserta hasil output di terminal

Command : `python solve_reverse.py`

3.4 Solver Script

```python
"""
GDGoC CTF - Reverse Engineering Challenge Solver
Binary: challenge (ELF 64-bit)

Binary ini menyembunyikan dua array byte di dalam main():
1. local_97 -> dipakai untuk MEMVALIDASI password (dibandingkan dengan input user)
               rumus: (input[i] ^ 0x39) == local_97[i]
2. local_b8 -> dipakai untuk MENGHASILKAN flag (hanya jalan kalau password benar)
               rumus: flag[i] = local_b8[i] ^ 0x5a

Script ini membalikkan kedua operasi XOR tersebut secara otomatis untuk
mendapatkan password dan flag, tanpa perlu menjalankan binary-nya.

Usage:
    python3 solve_reverse.py
"""

# Array ini diambil langsung dari hasil dekompilasi Ghidra (pseudo-code main()),
# persis sama angka-angka yang muncul di local_97[0] sampai local_97[0xd]
local_97 = [
    0x5e, 0x5d, 0x5e, 0x56, 0x5a, 0x66, 0x0b, 0x09,
    0x0b, 0x0d, 0x66, 0x5a, 0x4d, 0x5f,
]  # 14 bytes -> password check array

# Array ini juga diambil langsung dari Ghidra, dari local_b8[0] sampai local_b8[0x16]
local_b8 = [
    0x1d, 0x1e, 0x1d, 0x35, 0x19, 0x21, 0x28, 0x69,
    0x2c, 0x69, 0x28, 0x29, 0x69, 0x05, 0x69, 0x34,
    0x3d, 0x6b, 0x34, 0x69, 0x69, 0x28, 0x27,
]  # 23 bytes -> flag generator array

# Dua key ini diambil dari operator "^" yang muncul di pseudo-code
PASSWORD_XOR_KEY = 0x39  # dipakai buat validasi password
FLAG_XOR_KEY = 0x5a      # dipakai buat generate flag

def xor_decode(byte_array: list, key: int) -> str:
    """XOR every byte in the array with `key` and return as text."""
    # Tiap byte di array di-XOR sama key, hasilnya di-convert ke karakter
    # pakai chr(), lalu semua karakter digabung jadi satu string
    return "".join(chr(b ^ key) for b in byte_array)

def main():
    # Panggil xor_decode() buat dapetin password (dari local_97 + key 0x39)
    password = xor_decode(local_97, PASSWORD_XOR_KEY)

    # Panggil xor_decode() lagi buat dapetin flag (dari local_b8 + key 0x5a)
    flag = xor_decode(local_b8, FLAG_XOR_KEY)

    # Print hasil akhirnya
    print("Recovered password:", password)
    print("Recovered flag:    ", flag)

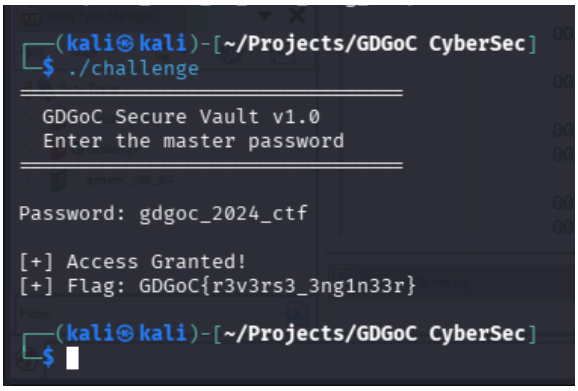
if __name__ == "__main__":
    main()
```

```
PS C:\Growth\python 2> python -u "c:\Growth\python 2\solve_reverse.py"
Recovered password: gdgoc_2024_ctf
Recovered flag:     GDGoC{r3v3rs3_3ng1n33r}
PS C:\Growth\python 2>
```

Gambar 3.4.1 - Script python solve_reverse

Command : `python solve_reverse.py`

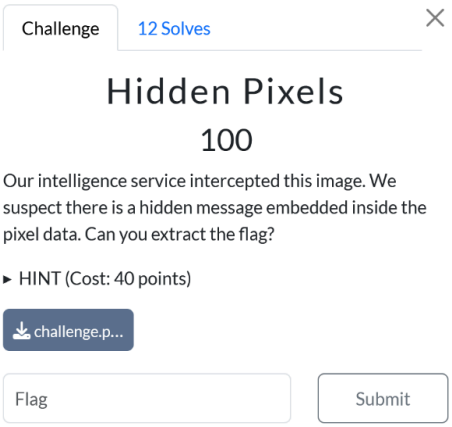
3.5 Flag



Gambar 3.5.1 - Flag challenge 3

Challenge 4 : Hidden Pixels (Forensics)

4.1 Challenge Description



Gambar 4.1 - Tampilan soal "Hidden Pixels" pada CTF platform

Challenge 4 meminta peserta untuk mencari pesan tersembunyi dari file challenge.png.

4.2 Methodology & Analysis

Concepts

Steganografi adalah teknik menyembunyikan suatu data atau pesan rahasia di dalam data lain (disebut *cover file*) sedemikian rupa sehingga keberadaan pesan tersebut tidak terlihat secara kasat mata, berbeda dengan kriptografi yang menyembunyikan **isi** pesan, steganografi menyembunyikan **keberadaan** pesan itu sendiri *(GeeksforGeeks, n.d.)*.

Salah satu teknik steganografi yang paling umum digunakan pada file gambar adalah **LSB (Least Significant Bit) steganography**. Setiap pixel pada gambar digital umumnya direpresentasikan dalam format RGB (Red, Green, Blue), di mana masing-masing channel warna disimpan dalam 1 byte (8 bit). Bit paling akhir (bit paling tidak signifikan/*least significant bit*) dari tiap byte dapat dimodifikasi untuk menyisipkan data rahasia, karena perubahan pada bit tersebut hanya mengubah nilai warna sebesar 1 tingkat dari 255 kemungkinan, sehingga tidak menimbulkan perubahan visual yang dapat dikenali oleh mata manusia *(GeeksforGeeks, n.d.)*.

Karena PNG merupakan format gambar yang bersifat *lossless* (tidak mengalami kompresi yang menghilangkan data), **teknik LSB steganography** dapat diterapkan dengan aman pada file PNG tanpa risiko data yang disisipkan menjadi rusak akibat proses kompresi.

Vulnerabilities

Berdasarkan konsep LSB steganography yang telah dijelaskan sebelumnya, saya mengidentifikasi bahwa kerentanan pada challenge ini terletak pada **penyisipan flag ke dalam least significant bit (LSB) dari channel warna Blue** pada file gambar `challenge.png`, tanpa disertai proteksi atau enkripsi tambahan terhadap data yang disisipkan.

Pada saat melakukan pengecekan awal menggunakan `exiftool`, ditemukan sebuah flag pada field metadata **Description**. Namun, setelah dianalisis lebih lanjut, flag tersebut merupakan **fake flag** yang sengaja disisipkan sebagai jebakan (decoy), sehingga diperlukan analisis lebih mendalam terhadap struktur data gambar itu sendiri, bukan hanya metadata-nya.

Dengan menggunakan tools `zsteg`, yang secara otomatis melakukan pengecekan kombinasi channel warna (Red, Green, Blue), urutan bit (LSB/MSB), serta arah pembacaan pixel, ditemukan bahwa flag asli tersimpan pada kombinasi:

`b1,b,lsb,xy`

Kombinasi tersebut menunjukkan bahwa flag disisipkan dengan mengambil 1 bit dari **channel Blue** pada setiap pixel, menggunakan **bit paling akhir (LSB)**, dan dibaca dengan urutan piksel dari kiri ke kanan, atas ke bawah (xy). Karena data disisipkan hanya pada 1 bit

LSB tanpa disertai enkripsi tambahan, flag dapat langsung diekstraksi dalam bentuk plaintext begitu kombinasi channel dan bit yang tepat ditemukan.

Dengan adanya kerentanan ini, penyerang dapat mengekstraksi flag secara langsung **tanpa perlu** melakukan brute-force maupun dekripsi tambahan, cukup dengan melakukan pengecekan menyeluruh terhadap seluruh kemungkinan kombinasi bit-plane pada gambar menggunakan tools yang sesuai.

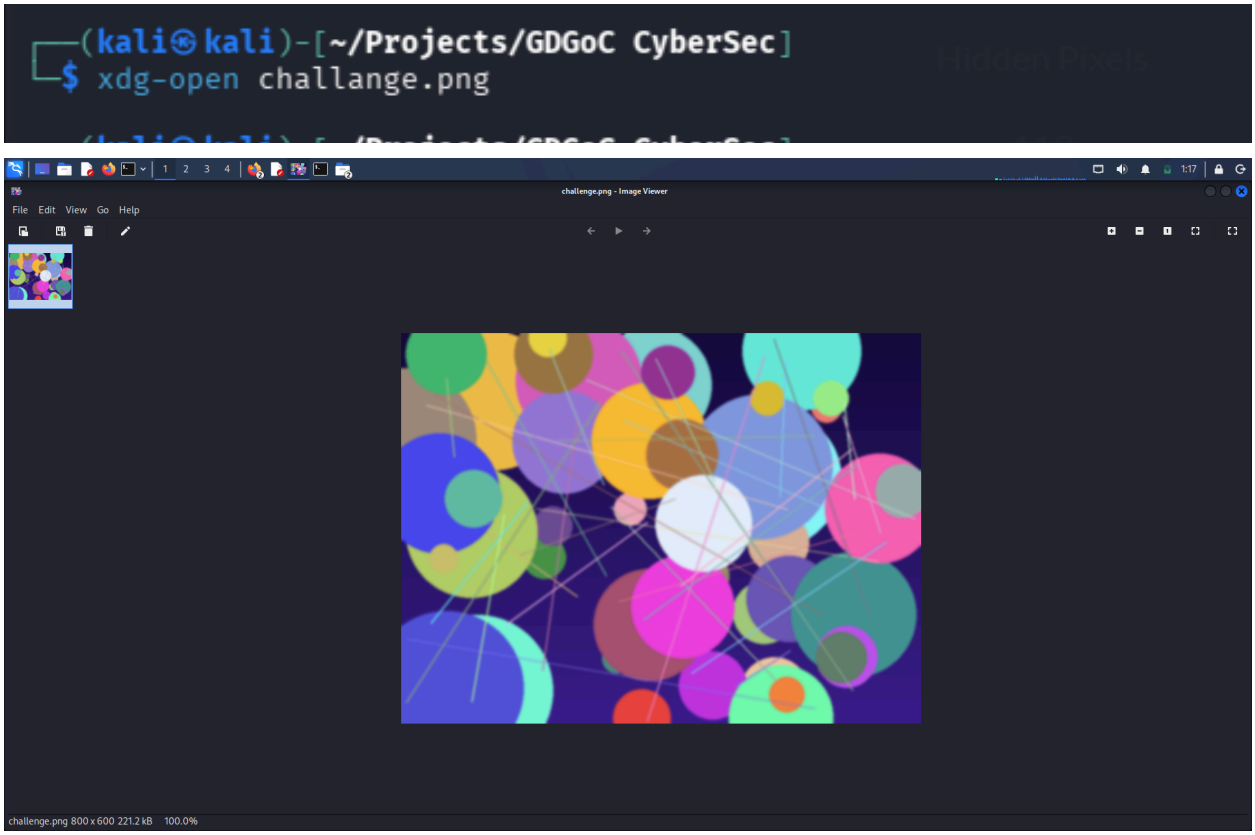
Logic

- Setelah mengunduh file `challenge.png`, saya menggunakan perintah `xdg-open challenge.png` terlebih dahulu untuk memastikan bahwa file dapat dibuka dan ditampilkan dengan normal sebagai gambar, guna memverifikasi bahwa file tersebut tidak korup dan sesuai dengan format PNG yang valid.
- Selanjutnya, saya melakukan pengecekan metadata menggunakan perintah `exiftool challenge.png` untuk melihat kemungkinan adanya informasi tersembunyi pada metadata gambar. Hasil pengecekan menunjukkan adanya sebuah flag pada field **Description**, namun setelah dianalisis lebih lanjut, flag tersebut merupakan **fake flag** yang sengaja disisipkan sebagai jebakan (decoy).
- Saya kemudian mencoba melakukan ekstraksi data tersembunyi menggunakan perintah `steghide extract -sf challenge.png`, namun muncul pesan error yang menyatakan bahwa format file PNG **tidak didukung** oleh steghide. Hal ini dikarenakan steghide hanya mendukung format file JPEG, BMP, WAV, dan AU, sehingga tidak dapat digunakan untuk melakukan ekstraksi pada file PNG.
- Oleh karena itu, saya beralih menggunakan tools `zsteg`, yang lebih sesuai untuk melakukan analisis **LSB steganography** pada file PNG. Tools ini diinstal menggunakan perintah `sudo apt update && sudo gem install zsteg` (karena zsteg merupakan Ruby gem, bukan package apt), kemudian dijalankan menggunakan perintah `zsteg -a challenge.png` untuk melakukan pengecekan menyeluruh terhadap seluruh kombinasi channel warna dan bit-plane. Untuk mempermudah pencarian, saya menyaring hasil output menggunakan `grep -i "GDGoC"`, sehingga ditemukan flag asli pada kombinasi `b1,b,lsb,xy`, yaitu 1 bit LSB pada channel Blue dengan urutan pembacaan piksel normal (kiri ke kanan, atas ke bawah).

4.3 Tools & Commands Used

- **Kali Linux Terminal** : untuk menjalankan command-line tools (xdg-open, exiftool, steghide, zsteg)

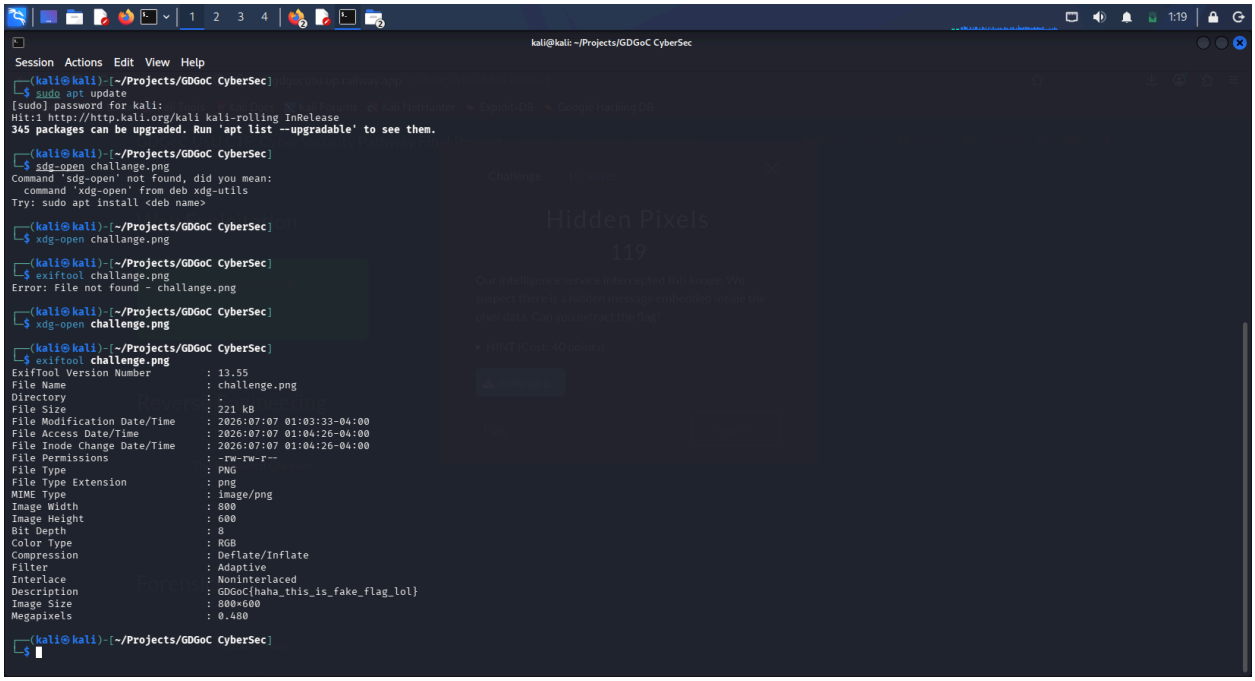
1. Membuka gambar menggunakan xdg-open



Gambar 4.3.1 - Tampilan challenge.png saat dibuka menggunakan xdg-open

Command : `xdg-open challenge.png`

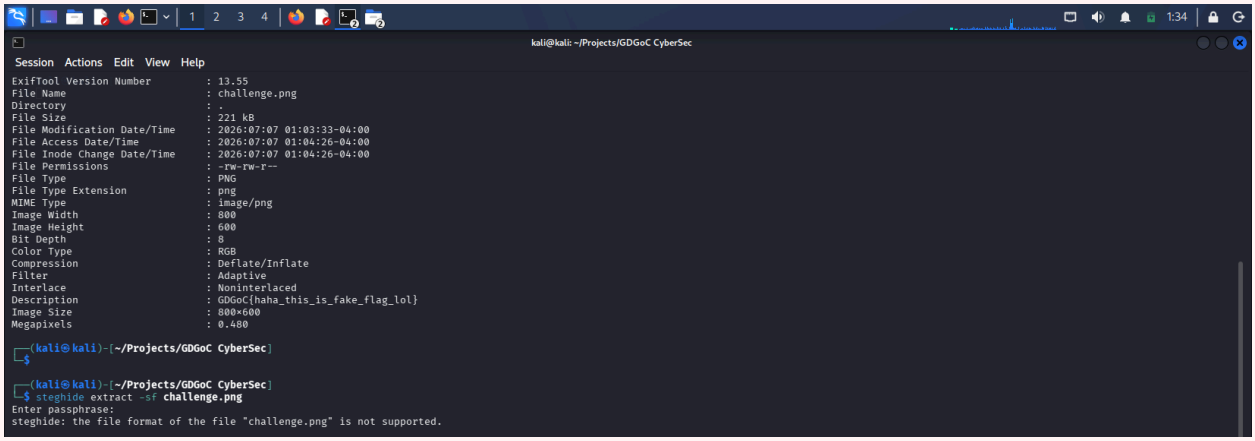
2. Cek metadata menggunakan exiftool



Gambar 4.3.2 - Hasil exiftool, ditemukan fake flag pada field Description

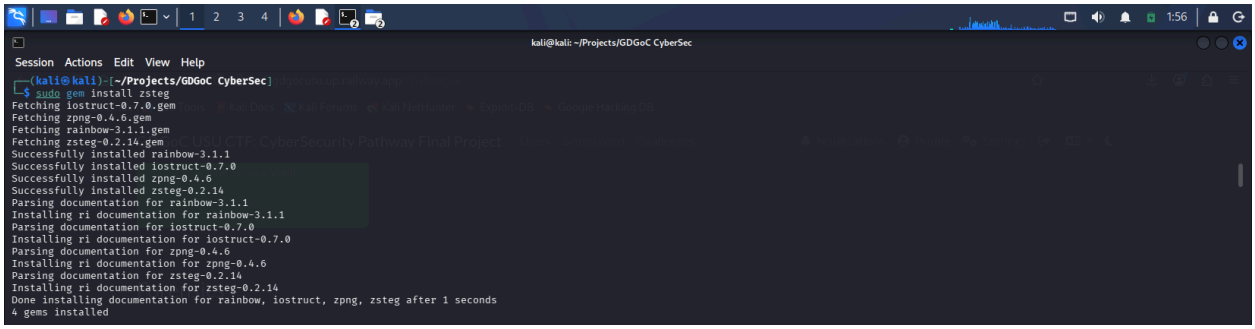
Command : `exiftool challenge.png`

3. Percobaan ekstraksi menggunakan steghide (gagal)



Gambar 4.3.3 - Error "file format not supported" saat menjalankan steghide pada file PNG

4. Instalasi zsteg



Gambar 4.3.4 - Proses instalasi zsteg menggunakan gem zsteg

Command : `sudo apt update`
`sudo gem install zsteg`

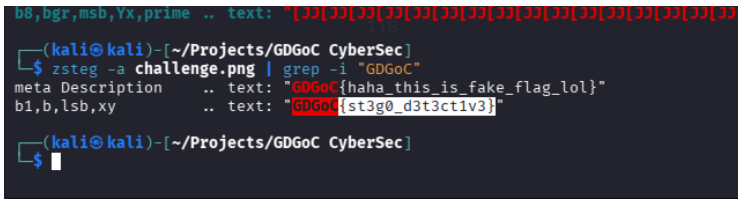
5. Hasil menemukan flag


(kali㉿kali)-[~/Projects/GDGoC CyberSec]
$ zsteg -a challenge.png
meta Description .. text: "GDGoC{haha_this_is_fake_flag_lol}"
chunk:0:IHDR .. file: Adobe Photoshop Color swatch, version 0, 800 colors; 1st RGB space (0), w 0×258, x 0×802, y 0, z 0; 2nd RGB space (0), w 0, x 0×100, y 0, z 0
b1,b,lsb,xy .. text: "GDGoC{st3g0_d3t3ct1v3}"
b1,bgr,lsb,xy .. file: 0420 Alliant virtual executable common library not stripped
b2,r,lsb,xy .. text: "UUUUUUUUUUUUUUUUL"
b2,g,lsb,xy .. text: ["U" repeated 24 times]
b2,g,msb,xy .. file: VISX image file
b2,b,lsb,xy .. text: "UUUUUUUUUUUUUZ"
b2,b,msb,xy .. text: ["U" repeated 10 times]
b3,b,lsb,xy .. file: OpenPGP Secret Key
b3p,r,msb,xy .. text: "\t\t\t

Gambar 4.3.5 - Hasil scan zsteg dengan filter grep menampilkan flag asli pada kombinasi b1,b,lsb,xy

Command : `zsteg -a challenge.png`
`zsteg -a challenge.png | grep -i “GDGoC”`

4.4 Flag



Gambar 4.4.1 - Flag challenge 4

Daftar Pustaka

FreeCodeCamp. (n.d.). *What is Base64 Encoding?*. FreeCodeCamp. https://www.freecodecamp.org/news/what-is-base64-encoding/

GeeksforGeeks. (n.d.). *Hexadecimal Number System*. GeeksforGeeks. https://www.geeksforgeeks.org/computer-organization-architecture/hexadecimal-number-system/

GeeksforGeeks. (n.d.). *How to Encrypt and Decrypt Strings in Python*. GeeksforGeeks. https://www.geeksforgeeks.org/python/how-to-encrypt-and-decrypt-strings-in-python/

GeeksforGeeks. (n.d.). *LSB Steganography*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/lsb-based-image-steganography-using-matlab/

GeeksforGeeks. (n.d.). *Reverse Engineering*. GeeksforGeeks. https://www.geeksforgeeks.org/software-engineering/software-engineering-reverse-engineering/

GeeksforGeeks. (n.d.). *What is Steganography?*. GeeksforGeeks. https://www.geeksforgeeks.org/cybersecurity/what-is-steganography/

Ghidra-SRE. (n.d.). *Ghidra Software Reverse Engineering Framework*. https://ghidra-sre.org/

OWASP Foundation. (n.d.). *SQL Injection*. OWASP. https://owasp.org/www-community/attacks/SQL_Injection

Wikipedia. (n.d.). *XOR cipher*. Wikipedia. https://en.wikipedia.org/wiki/XOR_cipher